1 The Task in Two Stages

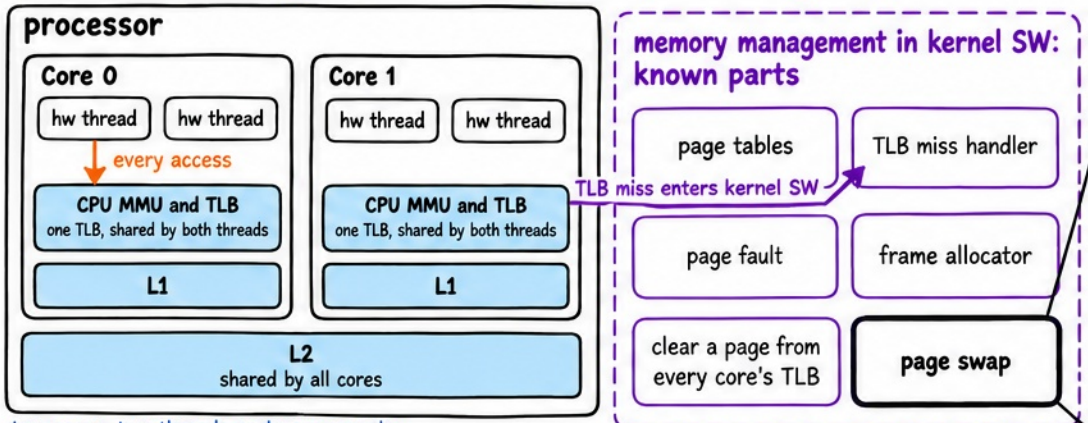
The CPU MMU and the memory management built on it are mature but new to us. Compression is a special case of one of its parts: page swap.

Stage 1: bring up the CPU MMU and memory management

MATURE DESIGN, NEW TO US

A SURE PATH TO THE END

the CPU MMU already exists in the chip: steady work brings it up



Our work

- ☐ integrate the CPU MMU and the caches into our system
- ☐ bring up each known part on this processor
- ☐ page tables shared by all cores, kept consistent with locks
- ☐ two threads of one core can miss on the same TLB at once
- ☐ clear a page from every core's TLB before reusing its frame
- ☐ keep L1 and L2 correct when a frame changes owner
- ☐ tests on every core and thread, not just one

Why we can expect it to work

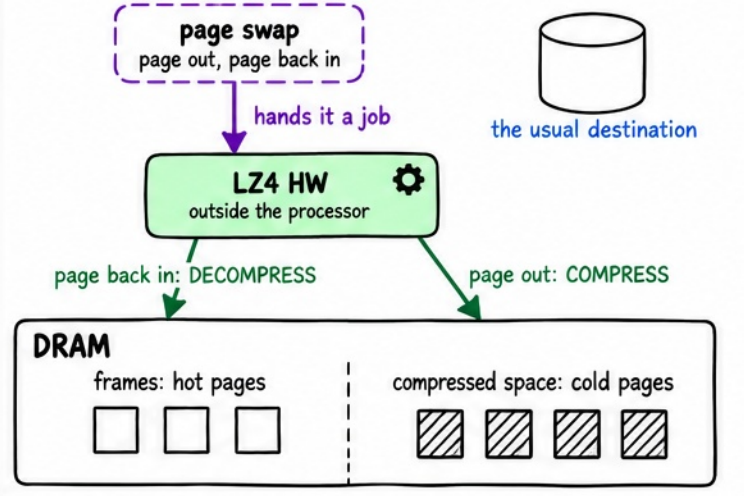
- ☐ the CPU MMU and these parts are a proven, mature design
- ☐ the hard part is that it is new to us, not the hardware

Stage 2: compression inside page swap

A SPECIAL CASE OF PAGE SWAP

SMALL SCOPE, MORE EXPLORATION

only one part of page swap, but its best design is still to be found



Our work

- ☐ LZ4 HW and the kernel SW that drives it
- ☐ manage the compressed space in DRAM
- ☐ choose which pages go cold
- ☐ tests: every page comes back unchanged

What stays, what is new

- ☐ stays: the page swap flow of page fault, page out, page back in
- ☐ new: the destination, compressed DRAM, and how it is managed

Key point Stage 1 brings up a mature CPU MMU and its memory management on many cores and threads. Stage 2 takes one of those parts, page swap, and gives it a new destination: compressed space in DRAM, moved by LZ4 HW.

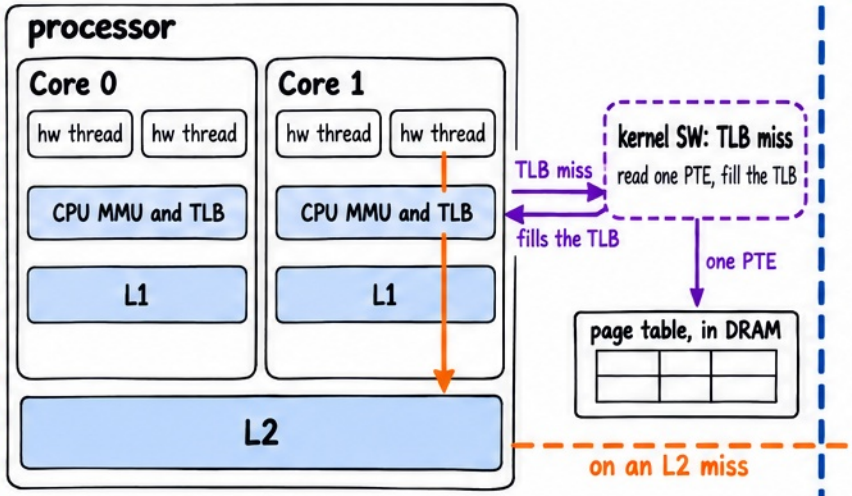
2 CPU MMU and Compression Layer: Apart in Space and Time

Two separate domains. The CPU MMU stands on every access inside each core. The compression layer stands beside DRAM and works only when a page swaps.

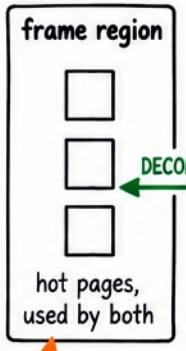
CPU MMU domain

ON THE ACCESS PATH

every load and store goes through the TLB of its core



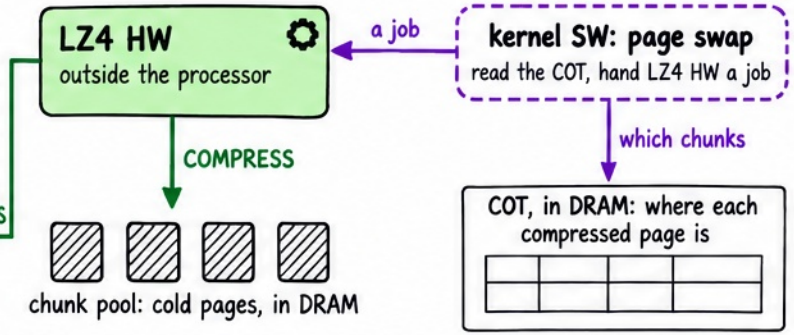
one kernel SW, two separate jobs and tables



compression domain

OFF THE ACCESS PATH

its table, the COT, is read only when a page swaps

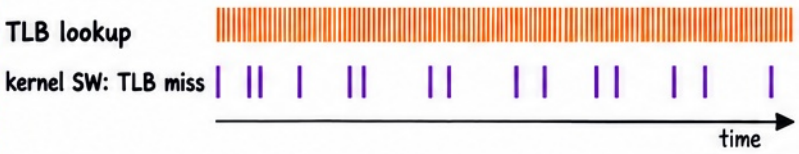


Rare, but each one counts

page swap happens only when the pages in use outgrow the frame region
each one takes locks, a COT lookup and an LZ4 HW job: budget the WCET

DENSE

every access; kernel SW on every TLB miss



SPARSE

only when a page swaps

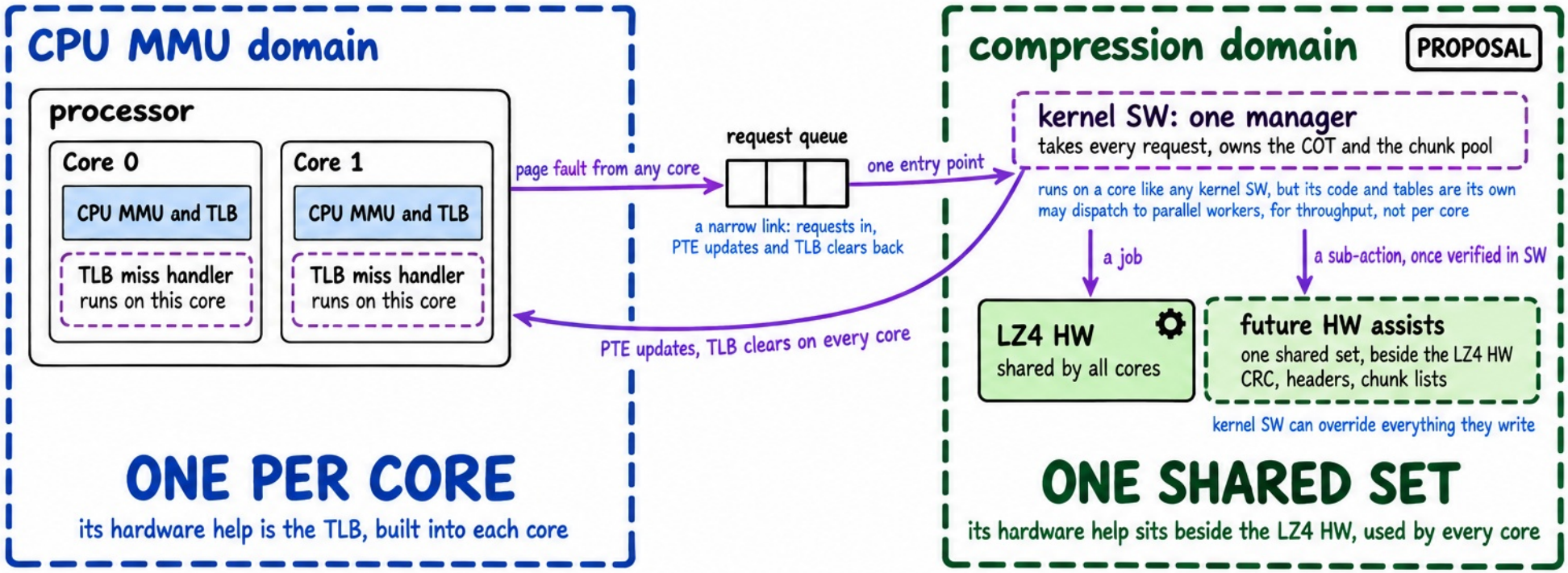


both timelines: the same stretch of time, spacing not measured

Key point The CPU MMU sits inside every core, on every access, and wakes kernel SW often. The compression layer sits outside, beside DRAM, and works only on page swap: rarely, but each time counts toward the worst case.

3 Apart Even in Software

The compression layer starts as pure kernel SW. That is no reason to design it together with the CPU MMU's kernel SW: the two grow in different places.



Where each one grows

	CPU MMU domain	compression domain
where the work runs	on the core that missed	in one manager, for all cores
where hardware help goes	inside each core: the TLB	one shared set beside the LZ4 HW
how it scales	one copy per core	more workers and engines, not per core

Why not co-design them

the CPU MMU side grows inside each core

the compression side grows in one shared place

design them together and a per-core path gets tied to a shared one.

Key point Even as pure kernel SW, the compression layer is designed apart from the CPU MMU's kernel SW: its future hardware help is one shared set beside the LZ4 HW, not something inside each core. The two meet only at a narrow interface.